LESSON 12 · CHEAT SHEET

Primitive Streams & Numeric Work

Java 8 → Java 21 · results use the shared course dataset
course "today" = 2026-11-07

Key rules

1. Three primitive streams only: `IntStream`, `LongStream`, `DoubleStream`. 8
2. `range(1, 5)` = 1..4 (half-open); `rangeClosed(1, 5)` = 1..5.
3. `sum()` of nothing = 0; `average/min/max` of nothing = **empty Optional**.
4. Need a Collector? `boxed()` or `mapToObj` first.
5. Boxed values are objects: compare with `equals`, never `==`.
6. **Money is never double.** BigDecimal (scale 2, HALF_UP) or `long` cents.
7. `chars()` gives ints (UTF-16 units); `codePoints()` gives real characters.

Canonical snippets

1 · Several statistics in one pass

```
flights.stream().mapToInt(Flight::seats)
       .summaryStatistics();
// count=8 sum=2078 min=160 max=471 avg=259.75
```

2 · Money total and average

```
var t = payments.stream().map(Payment::amount)
    .reduce(ZERO, BigDecimal::add);   // 4429.84
t.divide(valueOf(n), 2, HALF_UP);  // 369.15
```

3 · Character frequency

```
words.stream().flatMapToInt(String::chars)
     .mapToObj(c -> (char) c)
     .collect(groupingBy(identity(), TreeMap::new,
                         counting()));
```

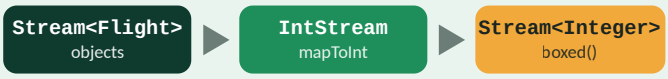
API map

Call	Gives	Since
`range / rangeClosed`	IntStream / LongStream	8
`iterate(seed, next).limit(n)`	infinite, then limited	8
`iterate(seed, hasNext, next)`	a for-loop as a stream	9
`mapToInt / mapToLong / mapToDouble`	object → primitive	8
`boxed() / mapToObj(f)`	primitive → object	8
`asLongStream / asDoubleStream`	widen, no cast	8
`sum()`	int / long / double	8
`average()`	OptionalDouble	8
`min() / max()`	OptionalInt / Long / Double	8
`summaryStatistics()`	Int/Long/DoubleSummaryStatistics	8
`String.chars() / codePoints()`	IntStream	8
`Random.ints(n, from, to)`	IntStream, to exclusive	8
`RandomGenerator.of(..)`	modern algorithms	17

double vs BigDecimal (real output)

```
0.1 + 0.2               0.30000000000000004
10 x 0.1 loop/reduce    0.9999999999999999
10 x 0.1 .sum()         1.0
0.1+0.2+0.3             0.6000000000000001
0.3+0.2+0.1             0.6
10 x 0.10 BigDecimal    1.00
```

Mini diagram: conversions



Real results from the course code

Call	Result
`IntStream.of(1000, 1000).boxed()`	== false, equals true (100: both true)
`"SIN".chars().boxed()`	[83, 73, 78]
"Pay " + card emoji	chars=6 codePoints=5
`new Random(42).ints(6, 1, 7)`	[3, 4, 1, 3, 1, 2] on every run

Worked example / exercise	Result
fullest flight	SQ21 SIN-EWR 159/161 98.76%
overall load	1808/2078 = 87.01%
empty seats	min=2 max=63 avg=33.75
flights ≥ 90% booked	[SQ21, SQ318, SQ938, TR808]

Anti-patterns → fixes

Don't	Do
`map(..).reduce(0, Integer::sum)`	`mapToInt(..).sum()`
`average().getAsDouble()`	`average().orElse(0)` or check
`a == b` on Integers	`a.equals(b)`

Don't	Do
`mapToDouble(amount).sum()`	`reduce(ZERO, BigDecimal::add)`
`divide(n)` without scale	`divide(n, 2, HALF_UP)`
`Random` for OTPs / tokens	`SecureRandom`

Lesson 12 · sticky notes

Cut out, stick on your wall, and read one every morning.

3 primitive streams

IntStream · LongStream
DoubleStream. No CharStream!

range is half-open

range(1, 5) → 1..4 = 10
rangeClosed(1, 5) → 15

Empty average

OptionalDouble.empty
sum() of nothing = 0

Never double for money

0.1 + 0.2 =
0.30000000000000004

Boxed == trap

1000 == 1000 → false
use equals()

One pass, 5 answers

summaryStatistics()
count sum min max avg

chars() gives ints

"SIN" → [83, 73, 78]
mapToObj(c -> (char) c)

Collector? Box first

boxed() / mapToObj
then groupingBy, toMap...